BANCO DE DADOS 2

Sistema de Centro Esportivo

Diagrama de classes e implementação em MySQL: seleção, inserção, atualização e remoção

Camila R. Ribeiro, Helena Vieira, Maria Clara Pastor
Prof. Raul E. S. Castanhari • IFSP Jacareí • ADS • 2026

O SISTEMA

Um centro esportivo com cinco entidades

Modalidades

Esportes oferecidos e preço mensal

Instrutores

Cada um leciona uma modalidade, com CREF único

Alunos

Cadastro com CPF único e data de matrícula

Matrículas

Liga alunos a modalidades, com data de adesão

Exames Médicos

Um exame por aluno: Apto ou Inapto

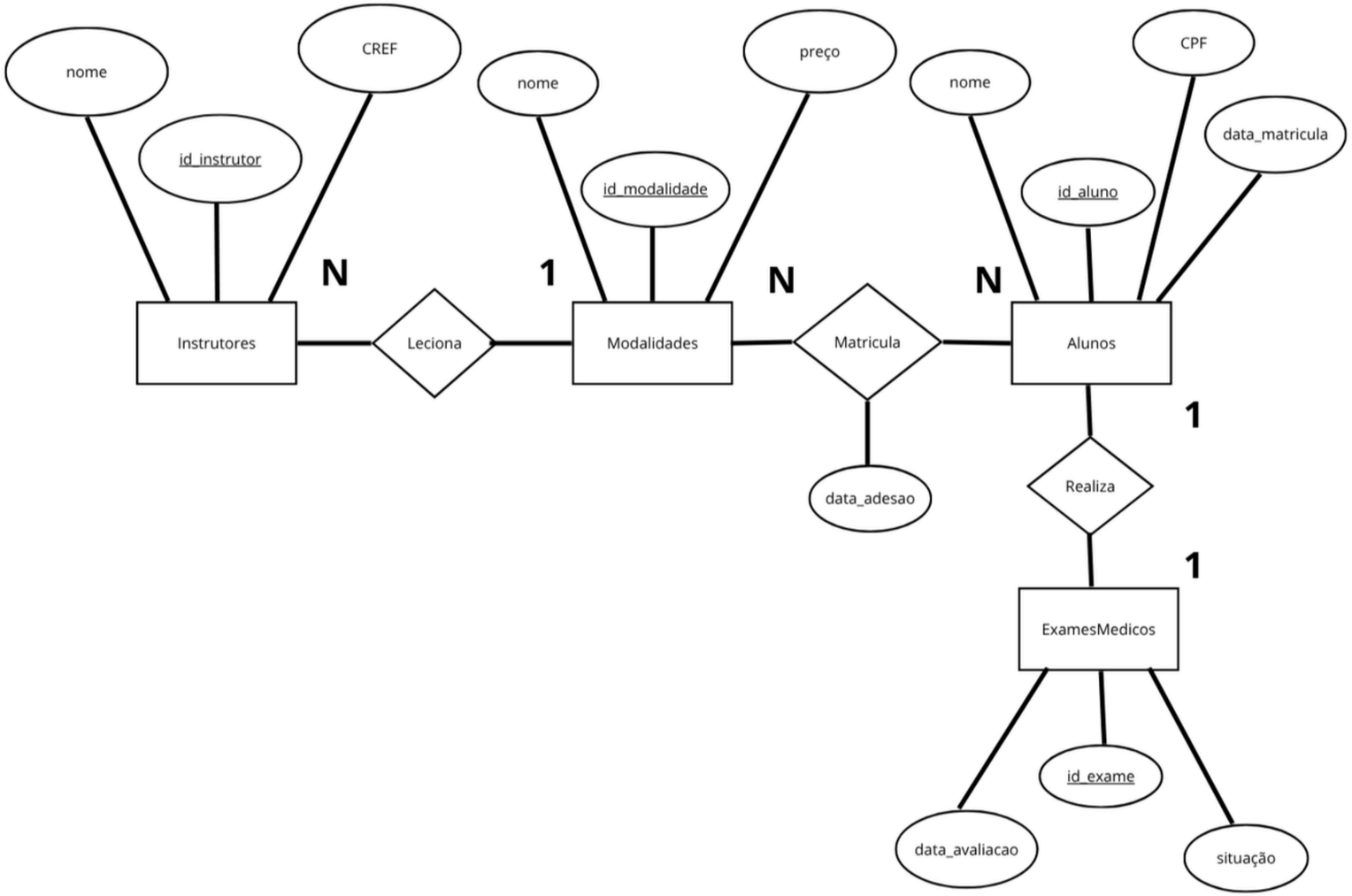
1:N Modalidades e Instrutores

N:N Alunos e Modalidades

1:1 Alunos e Exames

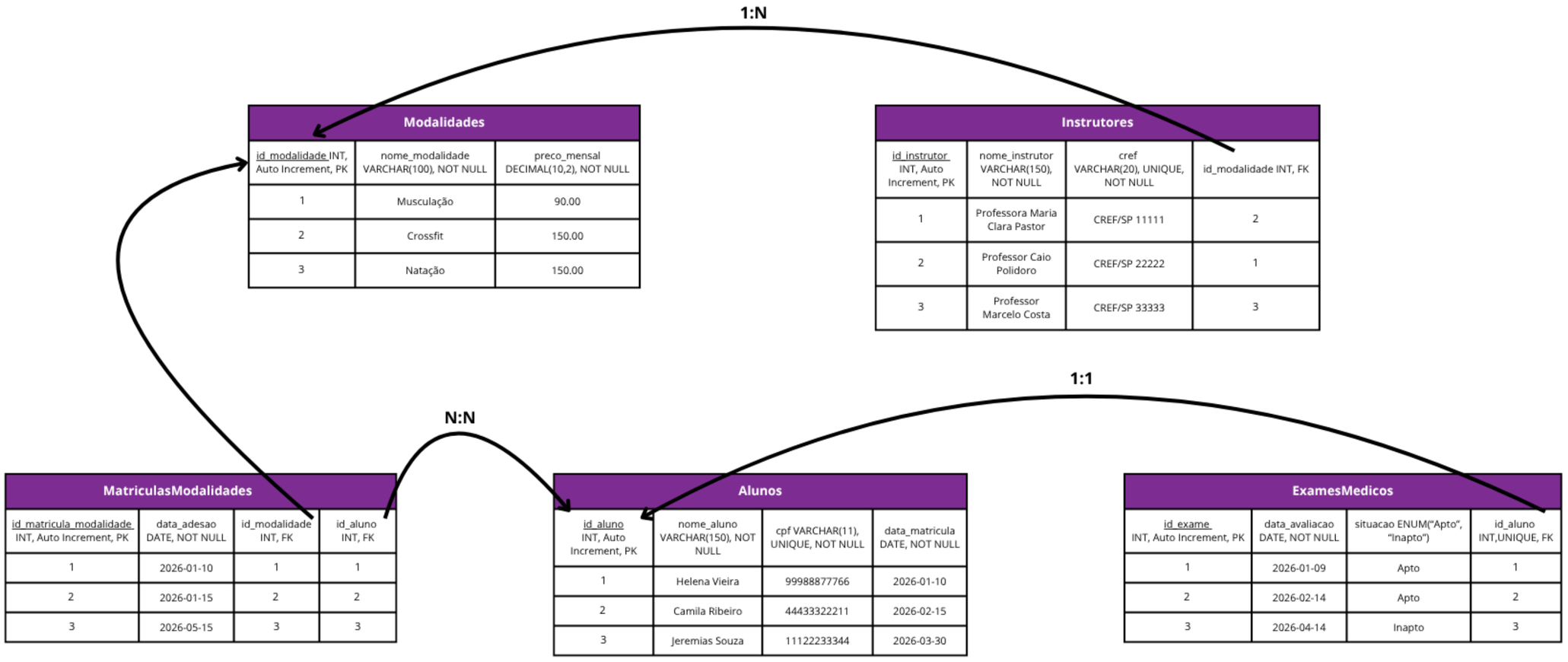
ARQUITETURA

Diagrama Entidade-Relacionamento Conceitual



ARQUITETURA

Modelo Lógico Relacional



Modalidades

id_modalidade INT, Auto Increment, PK	nome_modalidade VARCHAR(100), NOT NULL	preco_mensal DECIMAL(10,2), NOT NULL
1	Musculação	90.00
2	Crossfit	150.00
3	Natação	150.00

Instrutores

id_instrutor INT, Auto Increment, PK	nome_instrutor VARCHAR(150), NOT NULL	cref VARCHAR(20), UNIQUE, NOT NULL	id_modalidade INT, FK
1	Professora Maria Clara Pastor	CREF/SP 11111	2
2	Professor Caio Polidoro	CREF/SP 22222	1
3	Professor Marcelo Costa	CREF/SP 33333	3

MatriculasModalidades

id_matricula_modalidade INT, Auto Increment, PK	data_adesao DATE, NOT NULL	id_modalidade INT, FK	id_aluno INT, FK
1	2026-01-10	1	1
2	2026-01-15	2	2
3	2026-05-15	3	3

Alunos

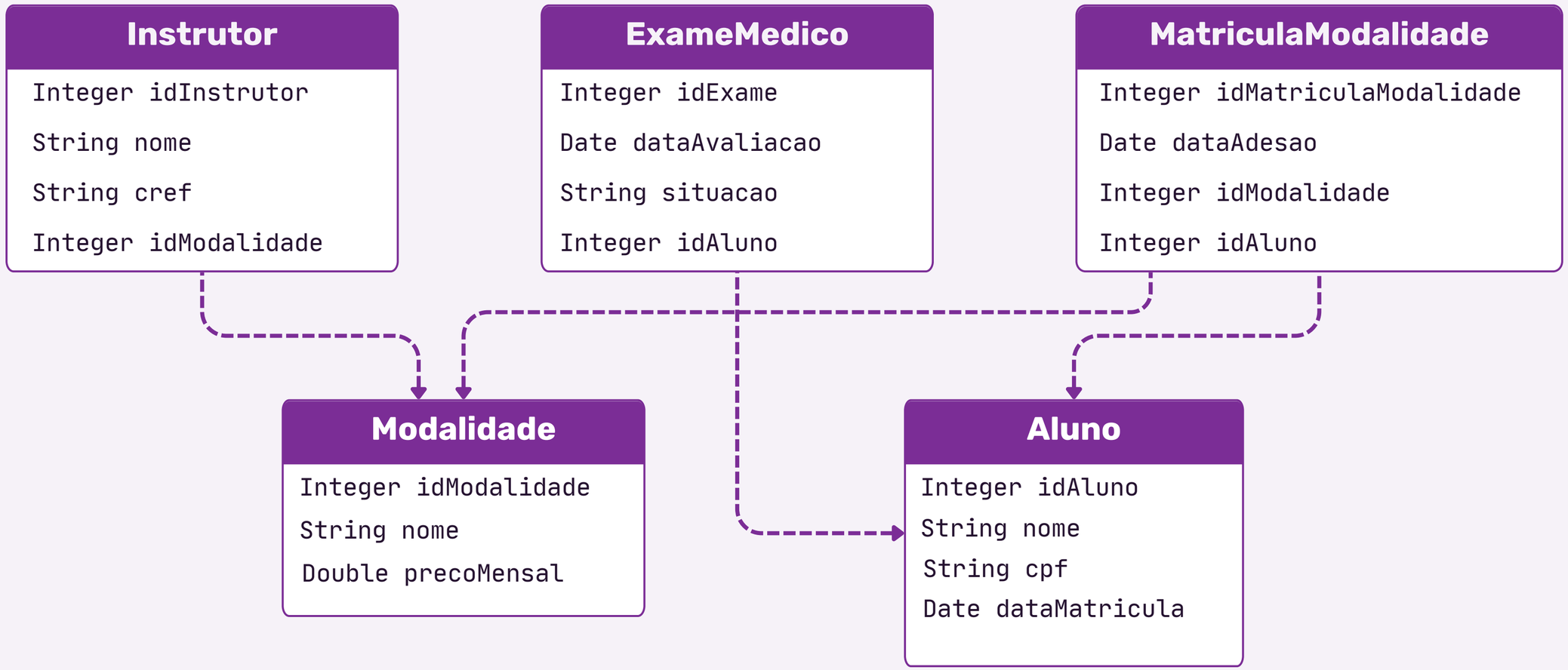
id_aluno INT, Auto Increment, PK	nome_aluno VARCHAR(150), NOT NULL	cpf VARCHAR(11), UNIQUE, NOT NULL	data_matricula DATE, NOT NULL
1	Helena Vieira	99988877766	2026-01-10
2	Camila Ribeiro	44433322211	2026-02-15
3	Jeremias Souza	11122233344	2026-03-30

ExamesMedicos

id_exame INT, Auto Increment, PK	data_avaliacao DATE, NOT NULL	situacao ENUM("Apto", "Inapto")	id_aluno INT,UNIQUE, FK
1	2026-01-09	Apto	1
2	2026-02-14	Apto	2
3	2026-04-14	Inapto	3

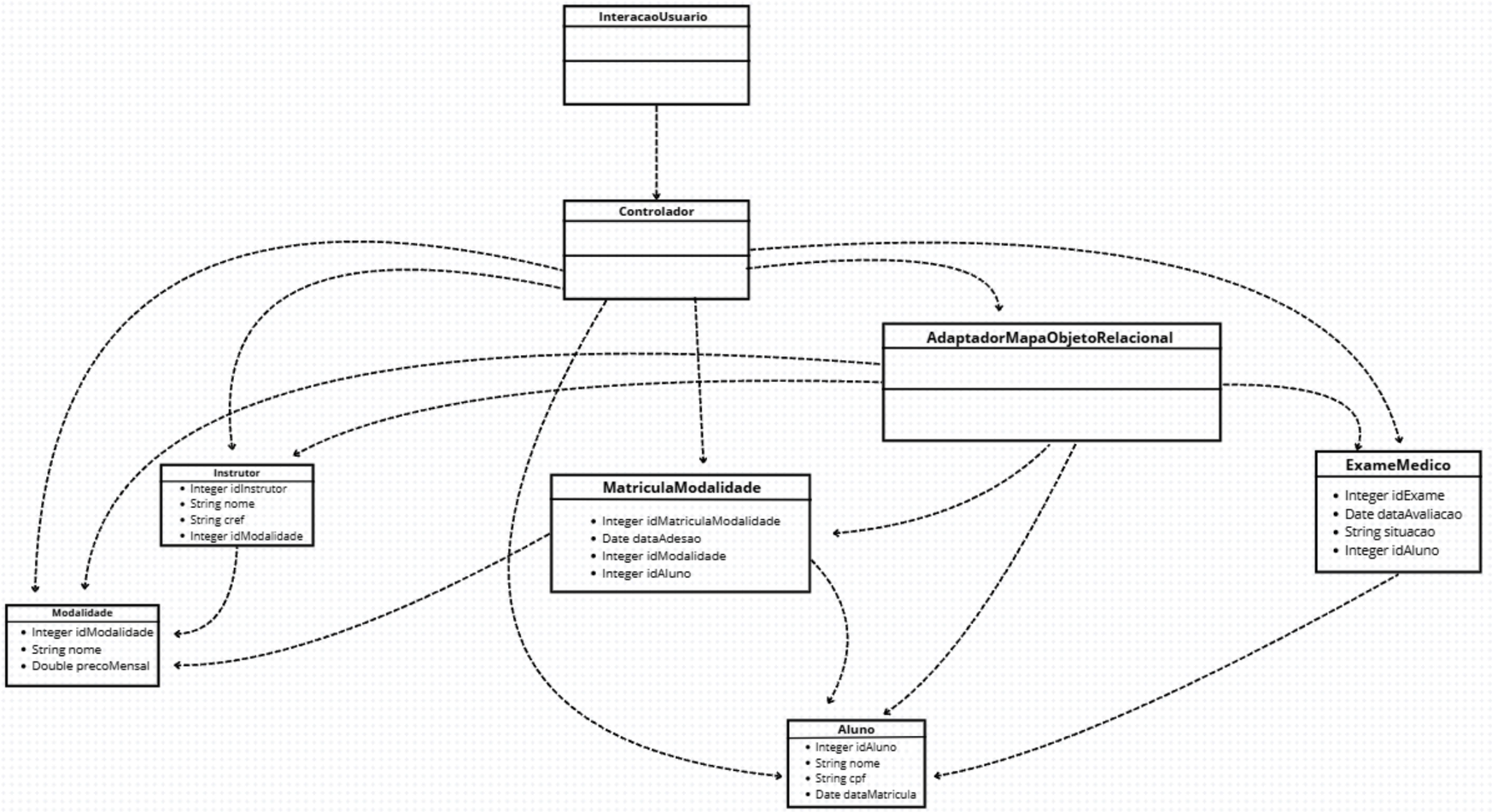
CLASSES DE DOMÍNIO

Diagrama de classes



ARQUITETURA

Persistência com ORM



MODELO RELACIONAL

Cinco tabelas e suas chaves

Tabela	Colunas	Chaves e restrições
Modalidades	id_modalidade, nome_modalidade, preco_mensal	PK id_modalidade
Instrutores	id_instrutor, nome_instrutor, cref, id_modalidade	PK id_instrutor • UNIQUE cref • FK id_modalidade
Alunos	id_aluno, nome_aluno, cpf, data_matricula	PK id_aluno • UNIQUE cpf
MatriculasModalidades	id_matricula_modalidade, data_adesao, id_modalidade, id_aluno	PK id_matricula_modalidade • FK id_modalidade e id_aluno
ExamesMedicos	id_exame, data_avaliacao, situacao, id_aluno	PK id_exame • FK e UNIQUE id_aluno

CÓDIGO FONTE • 1 DE 3

Criação do banco e tabelas base

```sql
DROP DATABASE IF EXISTS centro_esportivo;
CREATE DATABASE centro_esportivo
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE centro_esportivo;

CREATE TABLE Modalidades (
    id_modalidade INT AUTO_INCREMENT PRIMARY KEY,
    nome_modalidade VARCHAR(100) NOT NULL,
    preco_mensal DECIMAL(10, 2) NOT NULL,
    CONSTRAINT chk_preco_positivo CHECK (preco_mensal >= 0)
) ENGINE=InnoDB;

CREATE TABLE Alunos (
    id_aluno INT AUTO_INCREMENT PRIMARY KEY,
    nome_aluno VARCHAR(150) NOT NULL,
    cpf VARCHAR(11) NOT NULL UNIQUE,
    data_matricula DATE NOT NULL
) ENGINE=InnoDB;
```

Tabelas base

Modalidades e Alunos não dependem de outras tabelas.

Restrições

PK com AUTO_INCREMENT, CPF único e CHECK no preço.

CÓDIGO FONTE · 2 DE 3

Relações 1:N e 1:1

```
CREATE TABLE Instrutores (
    id_instrutor INT AUTO_INCREMENT PRIMARY KEY,
    nome_instrutor VARCHAR(150) NOT NULL,
    cref VARCHAR(20) NOT NULL UNIQUE,
    id_modalidade INT NOT NULL,
    CONSTRAINT fk_instrutor_modalidade
        FOREIGN KEY (id_modalidade) REFERENCES Modalidades(id_modalidade)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE ExamesMedicos (
    id_exame INT AUTO_INCREMENT PRIMARY KEY,
    data_avaliacao DATE NOT NULL,
    situacao ENUM('Apto', 'Inapto') NOT NULL,
    id_aluno INT NOT NULL UNIQUE,
    CONSTRAINT fk_exame_aluno
        FOREIGN KEY (id_aluno) REFERENCES Alunos(id_aluno)
        ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB;
```

1:N

Instrutor não pode ficar sem modalidade: ON DELETE RESTRICT.

1:1

UNIQUE em id_aluno garante um exame por aluno.

CÓDIGO FONTE • 3 DE 3

Relação N:N com tabela associativa

```
CREATE TABLE MatriculasModalidades (
    id_matricula_modalidade INT AUTO_INCREMENT PRIMARY KEY,
    data_adesao DATE NOT NULL,
    id_modalidade INT NOT NULL,
    id_aluno INT NOT NULL,
    CONSTRAINT fk_matricula_modalidade
        FOREIGN KEY (id_modalidade) REFERENCES Modalidades(id_modalidade)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_matricula_aluno
        FOREIGN KEY (id_aluno) REFERENCES Alunos(id_aluno)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT uq_aluno_modalidade UNIQUE (id_aluno, id_modalidade)
) ENGINE=InnoDB;
```

Duas chaves

Uma para Modalidades e outra para Alunos, ambas com CASCADE.

Sem duplicidade

UNIQUE no par impede o mesmo aluno na mesma modalidade duas vezes.

INSERÇÃO · INSERT

Carga de dados de exemplo

```
INSERT INTO Modalidades
    (id_modalidade, nome_modalidade, preco_mensal) VALUES
(1, 'Musculação', 90.00),
(2, 'Crossfit', 150.00),
(3, 'Natação', 150.00),
(4, 'Pilates', 130.00),
(5, 'Boxe', 110.00);

INSERT INTO MatriculasModalidades
    (data_adesao, id_modalidade, id_aluno) VALUES
('2026-01-10', 1, 1), ('2026-01-20', 2, 1),
('2026-02-15', 2, 2), ('2026-05-15', 3, 3),
('2026-04-10', 1, 4), ('2026-04-12', 3, 4),
('2026-05-15', 4, 5);
```

Volume

5 modalidades, 5 instrutores, 5 alunos, 7 matrículas e 5 exames.

SELEÇÃO COMPLEXA · 1 DE 4

Ficha de alunos aptos

```sql
SELECT
    a.nome_aluno AS Aluno,
    a.cpf AS CPF,
    m.nome_modalidade AS Modalidade,
    CONCAT('R$ ', FORMAT(m.preco_mensal, 2, 'pt_BR')) AS Mensalidade,
    i.nome_instrutor AS Instrutor_Responsavel,
    i.cref AS CREF,
    em.situacao AS Situacao_Medica,
    DATE_FORMAT(mm.data_adesao, '%d/%m/%Y') AS Data_Adesao
FROM Alunos a
INNER JOIN MatriculasModalidades mm ON a.id_aluno = mm.id_aluno
INNER JOIN Modalidades m ON mm.id_modalidade = m.id_modalidade
INNER JOIN Instrutores i ON m.id_modalidade = i.id_modalidade
INNER JOIN ExamesMedicos em ON a.id_aluno = em.id_aluno
WHERE em.situacao = 'Apto'
  AND m.preco_mensal >= 90.00
ORDER BY a.nome_aluno ASC, m.nome_modalidade ASC;
```

Junção de 5 tabelas

Alunos, matrículas, modalidades, instrutores e exames.

Filtros

Exame Apto e mensalidade de R$ 90 ou mais.

Resultado

6 linhas: cada aluno apto com sua modalidade e instrutor.

SELEÇÃO COMPLEXA · 2 DE 4

Auditoria médica: alunos inaptos

```sql
SELECT
    a.id_aluno,
    a.nome_aluno AS Aluno,
    em.situacao AS Status_Exame,
    DATE_FORMAT(em.data_avaliacao, '%d/%m/%Y') AS Data_Avaliacao_Medica,
    m.nome_modalidade AS Modalidade_Matriculada,
    i.nome_instrutor AS Instrutor_Avisado
FROM Alunos a
INNER JOIN ExamesMedicos em ON a.id_aluno = em.id_aluno
INNER JOIN MatriculasModalidades mm ON a.id_aluno = mm.id_aluno
INNER JOIN Modalidades m ON mm.id_modalidade = m.id_modalidade
INNER JOIN Instrutores i ON m.id_modalidade = i.id_modalidade
WHERE em.situacao = 'Inapto';
```

Para que serve

Achar quem está matriculado sem liberação médica.

Junção de 5 tabelas

Com filtro apenas em Inapto.

Resultado

1 linha: Jeremias Souza, Natação, Prof. Marcelo Costa.

SELEÇÃO COMPLEXA · 3 DE 4

Receita e alunos por modalidade

```
SELECT
    m.id_modalidade,
    m.nome_modalidade AS Modalidade,
    i.nome_instrutor AS Instrutor,
    m.preco_mensal AS Valor_Unitario,
    COUNT(mm.id_aluno) AS Total_Alunos_Matriculados,
    SUM(m.preco_mensal) AS Receita_Mensal_Gerada
FROM Modalidades m
LEFT JOIN Instrutores i ON m.id_modalidade = i.id_modalidade
LEFT JOIN MatriculasModalidades mm ON m.id_modalidade = mm.id_modalidade
GROUP BY m.id_modalidade, m.nome_modalidade,
    i.nome_instrutor, m.preco_mensal
HAVING COUNT(mm.id_aluno) > 0
ORDER BY Receita_Mensal_Gerada DESC;
```

Agregação

LEFT JOIN com GROUP BY, COUNT e SUM.

HAVING

Filtra o grupo: só modalidades com aluno.

Resultado

4 linhas: Crossfit e Natação com R$ 300 cada; Boxe fica de fora.

SELEÇÃO COMPLEXA · 4 DE 4

Alunos em mais de uma modalidade

```
SELECT
    a.id_aluno,
    a.nome_aluno AS Aluno,
    COUNT(mm.id_modalidade) AS Qtd_Modalidades,
    GROUP_CONCAT(m.nome_modalidade SEPARATOR ', ') AS Modalidades_Cursadas,
    SUM(m.preco_mensal) AS Mensalidade_Total_Aluno
FROM Alunos a
INNER JOIN MatriculasModalidades mm ON a.id_aluno = mm.id_aluno
INNER JOIN Modalidades m ON mm.id_modalidade = m.id_modalidade
GROUP BY a.id_aluno, a.nome_aluno
HAVING COUNT(mm.id_modalidade) > 1
ORDER BY Mensalidade_Total_Aluno DESC;
```

Agregação

Conta modalidades, junta os nomes e soma a mensalidade.

HAVING

Só quem pratica mais de um esporte.

Resultado

2 linhas: Helena e Lucas, cada um com R$ 240 por mês.

ATUALIZAÇÃO • UPDATE

Reajuste, liberação médica e cadastro

```sql
-- 1. Reajuste de 10% acima de R$ 100,00
UPDATE Modalidades
SET preco_mensal = ROUND(preco_mensal * 1.10, 2)
WHERE preco_mensal > 100.00;

-- 2. Liberação médica de Jeremias Souza
UPDATE ExamesMedicos
SET situacao = 'Apto',
    data_avaliacao = '2026-06-01'
WHERE id_aluno = 3;

-- 3. Atualização cadastral
UPDATE Alunos
SET nome_aluno = 'Helena Vieira da Silva'
WHERE cpf = '99988877766';
```

Reajuste

Crossfit e Natação vão a R$ 165, Pilates a R$ 143 e Boxe a R$ 121.

Liberação

Jeremias passa de Inapto para Apto.

Cadastro

Localiza a aluna pelo CPF, que é único.

REMOÇÃO · DELETE

Cancelamento e exclusão em cascata

```
-- 1. Cancela a matrícula de Jeremias em Natação
DELETE FROM MatriculasModalidades
WHERE id_aluno = 3 AND id_modalidade = 3;

-- 2. Remove a aluna Camila Ribeiro (id 2)
DELETE FROM Alunos
WHERE id_aluno = 2;

-- 3. Confere o efeito em cascata
SELECT * FROM MatriculasModalidades WHERE id_aluno = 2;
SELECT * FROM ExamesMedicos WHERE id_aluno = 2;
```

Cascata

Apagar o aluno remove suas matrículas e seu exame (ON DELETE CASCADE).

Verificação

As duas consultas finais voltam vazias: sem registros órfãos.

Proteção

Modalidade com instrutor não pode ser apagada (RESTRICT).

BACK END

Aplicação no terminal

Python

Menu interativo

Arquivo app.py, com menus de seleção feitos com a biblioteca questionary.

6 menus

CRUD e consultas

Um para cada entidade e um com as 4 consultas com JOIN.

CRUD

Operações completas

Listar, cadastrar, atualizar e excluir registros em cada menu.

Banco principal: MySQL

Reserva: SQLite

BACK END • CONEXÃO

Como o adaptador acha o banco

1

PyMySQL

Biblioteca Python para MySQL.

2

mysql-connector

Driver oficial, se o primeiro não existir.

3

Cliente mysql

Programa de linha de comando do MySQL.

4

SQLite

Reserva: arquivo centro_esportivo.db.

Ao conectar, o adaptador cria banco e tabelas se não existirem e carrega dados iniciais quando Modalidades está vazia.

Obrigada!

REPOSITÓRIO NO GITHUB:

https://github.com/camilaribeirox/centro-esportivo.git

Dúvidas?